

Bước 0: Thiết lập Môi trường và Tải Dữ liệu

```
!pip install gensim

Requirement already satisfied: gensim in
/usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in
/usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in
/usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in
/usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in
/usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1-
>gensim) (2.0.1)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report, f1_score,
accuracy_score

from gensim.models import Word2Vec

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Embedding, LSTM
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import to_categorical

print(f"GPU Available: {tf.config.list_physical_devices('GPU')}")

GPU Available: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]

train_path = '/content/train.csv'
val_path = '/content/val.csv'
test_path = '/content/test.csv'

# Đọc dữ liệu
df_train = pd.read_csv(train_path)
```

```

df_val = pd.read_csv(val_path)
df_test = pd.read_csv(test_path)

print("Train shape:", df_train.shape)
print("Validation shape:", df_val.shape)
print("Test shape:", df_test.shape)

df_train.head()

Train shape: (8954, 2)
Validation shape: (1076, 2)
Test shape: (1076, 2)

{"summary":{"\n  \"name\": \"df_train\",\n  \"rows\": 8954,\n  \"fields\": [\n    {\n      \"column\": \"text\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 8954,\n        \"samples\": [\n          \"please show me the list that i have\",\n          \"raise lights to full power\",\n          \"i think you made some mistake, please check it.\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          }\n        },\n        {\n          \"column\": \"category\",\n          \"properties\": {\n            \"dtype\": \"category\",\n            \"num_unique_values\": 64,\n            \"samples\": [\n              \"recommendation_events\",\n              \"takeaway_query\",\n              \"alarm_query\",\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            }\n          }\n        ]\n      },\n      \"type\": \"dataframe\", \"variable_name\": \"df_train\"}

print("Columns:", df_train.columns.tolist())
print("\nSample data:")
print(df_train.head())

print("\nNumber of unique categories:",
df_train['category'].nunique())
print("\nCategory distribution:")
print(df_train['category'].value_counts().head(10))

Columns: ['text', 'category']

Sample data:

```

	text	category
0	what alarms do i have set right now	alarm_query
1	checkout today alarm of meeting	alarm_query
2	report alarm settings	alarm_query
3	see see for me the alarms that you have set to...	alarm_query
4	is there an alarm for ten am	alarm_query

```

Number of unique categories: 64

Category distribution:
category

```

```
alarm_set          159
social_post        159
qa_definition      159
alarm_query        158
general_quirky     158
iot_hue_lightchange 158
calendar_query     158
general_commandstop 158
lists_query        158
lists_createoradd  158
Name: count, dtype: int64
```

Tiền xử lý các nhãn (intent) để chuyển chúng thành dạng số

```
# Encode labels
label_encoder = LabelEncoder()
label_encoder.fit(pd.concat([df_train['category'], df_val['category'],
df_test['category']]))

y_train = label_encoder.transform(df_train['category'])
y_val = label_encoder.transform(df_val['category'])
y_test = label_encoder.transform(df_test['category'])

num_classes = len(label_encoder.classes_)
print(f"Number of classes: {num_classes}")
print(f"Classes: {label_encoder.classes_[:10]}\n")

X_train_text = df_train['text'].values
X_val_text = df_val['text'].values
X_test_text = df_test['text'].values

print(f"Train samples: {len(X_train_text)}")
print(f"Validation samples: {len(X_val_text)}")
print(f"Test samples: {len(X_test_text)}")

Number of classes: 64
Classes: ['alarm_query' 'alarm_remove' 'alarm_set' 'audio_volume_down'
'audio_volume_mute' 'audio_volume_up' 'calendar_query'
'calendar_remove'
'calendar_set' 'cooking_recipe']

Train samples: 8954
Validation samples: 1076
Test samples: 1076
```

Nhiệm vụ 1: (Warm-up Ôn bài cũ) Pipeline TF-IDF + Logistic Regression

```
tfidf_log_reg_pipeline = make_pipeline(
    TfidfVectorizer(max_features=5000, ngram_range=(1, 2)),
    LogisticRegression(max_iter=1000, random_state=2, n_jobs=-1)
)

print("\nTraining TF-IDF + Logistic Regression...")
tfidf_log_reg_pipeline.fit(X_train_text, y_train)

y_pred_tfidf_log_reg = tfidf_log_reg_pipeline.predict(X_test_text)

accuracy_tfidf_log_reg = accuracy_score(y_test, y_pred_tfidf_log_reg)
f1_macro_tfidf_log_reg = f1_score(y_test, y_pred_tfidf_log_reg,
average='macro')

print(f"\nAccuracy: {accuracy_tfidf_log_reg:.4f}")
print(f"F1-score (Macro): {f1_macro_tfidf_log_reg:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_tfidf_log_reg,
                           target_names=label_encoder.classes_,
                           digits=4))
```

Training TF-IDF + Logistic Regression...

Accuracy: 0.8336
F1-score (Macro): 0.8300

Classification Report:

	precision	recall	f1-score	support
alarm_query	0.9474	0.9474	0.9474	19
alarm_remove	1.0000	0.7273	0.8421	11
alarm_set	0.8500	0.8947	0.8718	19
audio_volume_down	1.0000	0.7500	0.8571	8
audio_volume_mute	0.9231	0.8000	0.8571	15
audio_volume_up	1.0000	1.0000	1.0000	13
calendar_query	0.5789	0.5789	0.5789	19
calendar_remove	0.7826	0.9474	0.8571	19
calendar_set	0.8750	0.7368	0.8000	19
cooking_recipe	0.9286	0.6842	0.7879	19
datetime_convert	0.7778	0.8750	0.8235	8
datetime_query	0.7083	0.8947	0.7907	19
email_addcontact	0.8750	0.8750	0.8750	8
email_query	0.8333	0.7895	0.8108	19
email_querycontact	0.9286	0.6842	0.7879	19
email_sendemail	0.7727	0.8947	0.8293	19

general_affirm	0.9500	1.0000	0.9744	19
general_commandstop	1.0000	1.0000	1.0000	19
general_confirm	1.0000	1.0000	1.0000	19
general_dontcare	0.9048	1.0000	0.9500	19
general_explain	1.0000	1.0000	1.0000	19
general_joke	1.0000	1.0000	1.0000	12
general_negate	1.0000	0.9474	0.9730	19
general_praise	0.9500	1.0000	0.9744	19
general_quirky	0.3333	0.2632	0.2941	19
general_repeat	0.9500	1.0000	0.9744	19
iot_cleaning	1.0000	1.0000	1.0000	16
iot_coffee	0.9000	0.9474	0.9231	19
iot_hue_lightchange	0.7895	0.7895	0.7895	19
iot_hue_lightdim	0.9091	0.8333	0.8696	12
iot_hue_lightoff	0.8571	0.9474	0.9000	19
iot_hue_lighton	0.4000	0.6667	0.5000	3
iot_hue_lightup	0.9286	0.9286	0.9286	14
iot_wemo_off	0.8000	0.8889	0.8421	9
iot_wemo_on	0.6250	0.7143	0.6667	7
lists_createoradd	0.6522	0.7895	0.7143	19
lists_query	0.7895	0.7895	0.7895	19
lists_remove	0.8571	0.9474	0.9000	19
music_likeness	0.5714	0.6667	0.6154	18
music_query	0.6875	0.5789	0.6286	19
music_settings	1.0000	0.4286	0.6000	7
news_query	0.8571	0.6316	0.7273	19
play_audiobook	0.9000	0.9474	0.9231	19
play_game	0.8462	0.5789	0.6875	19
play_music	0.5000	0.6316	0.5581	19
play_podcasts	1.0000	0.8421	0.9143	19
play_radio	0.9333	0.7368	0.8235	19
qa_currency	0.8947	0.8947	0.8947	19
qa_definition	0.9474	0.9474	0.9474	19
qa_factoid	0.4074	0.5789	0.4783	19
qa_maths	0.9286	0.9286	0.9286	14
qa_stock	0.9474	0.9474	0.9474	19
recommendation_events	0.7500	0.7895	0.7692	19
recommendation_locations	0.8000	0.8421	0.8205	19
recommendation_movies	0.9091	1.0000	0.9524	10
social_post	0.9474	0.9474	0.9474	19
social_query	0.7895	0.8333	0.8108	18
takeaway_order	0.7647	0.6842	0.7222	19
takeaway_query	0.8947	0.8947	0.8947	19
transport_query	0.6957	0.8421	0.7619	19
transport_taxi	1.0000	1.0000	1.0000	18
transport_ticket	0.9375	0.7895	0.8571	19
transport_traffic	0.9474	0.9474	0.9474	19
weather_query	0.6842	0.6842	0.6842	19

accuracy			0.8336	1076
macro avg	0.8425	0.8301	0.8300	1076
weighted avg	0.8434	0.8336	0.8337	1076

Nhiệm vụ 2: (Warm-up Ôn bài cũ) Pipeline Word2Vec (Trung bình) + Dense Layer

```
sentences_train = [text.lower().split() for text in X_train_text]
sentences_val = [text.lower().split() for text in X_val_text]
sentences_test = [text.lower().split() for text in X_test_text]

print("\nTraining Word2Vec model")
w2v_model = Word2Vec(
    sentences=sentences_train,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4,
    epochs=10,
    seed=42
)

print(f"Vocabulary size: {len(w2v_model.wv)}")
print(f"Vector size: {w2v_model.vector_size}")

def sentence_to_avg_vector(sentence, model):
    words = sentence.lower().split()
    word_vectors = []

    for word in words:
        if word in model.wv:
            word_vectors.append(model.wv[word])

    if len(word_vectors) > 0:
        return np.mean(word_vectors, axis=0)
    else:
        return np.zeros(model.vector_size)

print("Converting sentences to average vectors")
X_train_w2v = np.array([sentence_to_avg_vector(text, w2v_model) for
text in X_train_text])
X_val_w2v = np.array([sentence_to_avg_vector(text, w2v_model) for text
in X_val_text])
X_test_w2v = np.array([sentence_to_avg_vector(text, w2v_model) for
text in X_test_text])
```

```
print(f"Data training Word2Vec shape: {X_train_w2v.shape}")
print(f"Data validation Word2Vec shape: {X_val_w2v.shape}")
print(f"Data testing Word2Vec shape: {X_test_w2v.shape}")
```

Training Word2Vec model

Vocabulary size: 4424

Vector size: 100

Converting sentences to average vectors

Data training Word2Vec shape: (8954, 100)

Data validation Word2Vec shape: (1076, 100)

Data testing Word2Vec shape: (1076, 100)

```
y_train_processed = to_categorical(y_train, num_classes)
```

```
y_val_processed = to_categorical(y_val, num_classes)
```

```
y_test_processed = to_categorical(y_test, num_classes)
```

```
from tensorflow.keras.layers import BatchNormalization
```

```
from tensorflow.keras.optimizers import Adam
```

```
print("\nBuilding Word2Vec + Dense model")
```

```
word2vec_dense_model = Sequential([
    Dense(256, activation='relu',
input_shape=(w2v_model.vector_size,)),
    BatchNormalization(),
    Dropout(0.4),

    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),

    Dense(64, activation='relu'),
    BatchNormalization(),
    Dropout(0.25),

    Dense(num_classes, activation='softmax')
])
```

```
word2vec_dense_model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
word2vec_dense_model.summary()
```

Building Word2Vec + Dense model

Model: "sequential_5"

Layer (type)	Output Shape
Param #	
dense_11 (Dense)	(None, 256)
25,856	
batch_normalization_6	(None, 256)
1,024	
(BatchNormalization)	
dropout_6 (Dropout)	(None, 256)
0	
dense_12 (Dense)	(None, 128)
32,896	
batch_normalization_7	(None, 128)
512	
(BatchNormalization)	
dropout_7 (Dropout)	(None, 128)
0	
dense_13 (Dense)	(None, 64)
8,256	
batch_normalization_8	(None, 64)
256	
(BatchNormalization)	
dropout_8 (Dropout)	(None, 64)
0	
dense_14 (Dense)	(None, 64)
4,160	


```
Total params: 72,960 (285.00 KB)
```

```
Trainable params: 72,064 (281.50 KB)
```

```
Non-trainable params: 896 (3.50 KB)
```

```
early_stop = EarlyStopping(monitor='val_loss', patience=20,  
restore_best_weights=True, verbose=1)
```

```
history_word2vec = word2vec_dense_model.fit(  
    X_train_w2v, y_train_processed,  
    validation_data=(X_val_w2v, y_val_processed),  
    epochs=100,  
    batch_size=32,  
    callbacks=[early_stop],  
    verbose=1  
)
```

```
Epoch 1/100
```

```
280/280 _____ 21s 32ms/step - accuracy: 0.0575 - loss:  
4.2542 - val_accuracy: 0.1599 - val_loss: 3.6511
```

```
Epoch 2/100
```

```
280/280 _____ 1s 3ms/step - accuracy: 0.1706 - loss:  
3.2813 - val_accuracy: 0.2853 - val_loss: 2.7894
```

```
Epoch 3/100
```

```
280/280 _____ 1s 3ms/step - accuracy: 0.2302 - loss:  
2.9227 - val_accuracy: 0.3578 - val_loss: 2.4579
```

```
Epoch 4/100
```

```
280/280 _____ 1s 3ms/step - accuracy: 0.2855 - loss:  
2.6774 - val_accuracy: 0.3820 - val_loss: 2.2950
```

```
Epoch 5/100
```

```
280/280 _____ 1s 3ms/step - accuracy: 0.3087 - loss:  
2.5426 - val_accuracy: 0.4089 - val_loss: 2.1803
```

```
Epoch 6/100
```

```
280/280 _____ 1s 3ms/step - accuracy: 0.3342 - loss:  
2.4212 - val_accuracy: 0.4201 - val_loss: 2.0892
```

```
Epoch 7/100
```

```
280/280 _____ 1s 3ms/step - accuracy: 0.3450 - loss:  
2.3572 - val_accuracy: 0.4247 - val_loss: 2.0714
```

```
Epoch 8/100
```

```
280/280 _____ 1s 4ms/step - accuracy: 0.3858 - loss:  
2.2398 - val_accuracy: 0.4610 - val_loss: 2.0114
```

```
Epoch 9/100
```

```
280/280 _____ 1s 4ms/step - accuracy: 0.3862 - loss:  
2.1978 - val_accuracy: 0.4387 - val_loss: 2.0441
```

```
Epoch 10/100
```

```
280/280 _____ 1s 4ms/step - accuracy: 0.4012 - loss:
```

2.1199 - val_accuracy: 0.4480 - val_loss: 1.9352
Epoch 11/100
280/280 _____ 1s 3ms/step - accuracy: 0.4155 - loss: 2.1176 - val_accuracy: 0.4582 - val_loss: 1.8974
Epoch 12/100
280/280 _____ 1s 3ms/step - accuracy: 0.4134 - loss: 2.0852 - val_accuracy: 0.4888 - val_loss: 1.8294
Epoch 13/100
280/280 _____ 1s 3ms/step - accuracy: 0.4242 - loss: 2.0494 - val_accuracy: 0.5056 - val_loss: 1.7481
Epoch 14/100
280/280 _____ 1s 3ms/step - accuracy: 0.4415 - loss: 1.9827 - val_accuracy: 0.4870 - val_loss: 1.7471
Epoch 15/100
280/280 _____ 1s 3ms/step - accuracy: 0.4483 - loss: 1.9583 - val_accuracy: 0.4898 - val_loss: 1.7626
Epoch 16/100
280/280 _____ 1s 3ms/step - accuracy: 0.4382 - loss: 1.9829 - val_accuracy: 0.5353 - val_loss: 1.6684
Epoch 17/100
280/280 _____ 1s 3ms/step - accuracy: 0.4527 - loss: 1.9532 - val_accuracy: 0.5121 - val_loss: 1.7164
Epoch 18/100
280/280 _____ 1s 3ms/step - accuracy: 0.4598 - loss: 1.9223 - val_accuracy: 0.5009 - val_loss: 1.7360
Epoch 19/100
280/280 _____ 1s 3ms/step - accuracy: 0.4676 - loss: 1.8762 - val_accuracy: 0.4740 - val_loss: 1.8375
Epoch 20/100
280/280 _____ 1s 3ms/step - accuracy: 0.4782 - loss: 1.8712 - val_accuracy: 0.5028 - val_loss: 1.7895
Epoch 21/100
280/280 _____ 1s 4ms/step - accuracy: 0.4699 - loss: 1.8558 - val_accuracy: 0.4749 - val_loss: 1.9069
Epoch 22/100
280/280 _____ 1s 4ms/step - accuracy: 0.4681 - loss: 1.8837 - val_accuracy: 0.5232 - val_loss: 1.6265
Epoch 23/100
280/280 _____ 1s 4ms/step - accuracy: 0.4827 - loss: 1.8368 - val_accuracy: 0.5149 - val_loss: 1.6788
Epoch 24/100
280/280 _____ 1s 3ms/step - accuracy: 0.5022 - loss: 1.7654 - val_accuracy: 0.5112 - val_loss: 1.6785
Epoch 25/100
280/280 _____ 1s 3ms/step - accuracy: 0.4870 - loss: 1.8101 - val_accuracy: 0.4972 - val_loss: 1.8431
Epoch 26/100
280/280 _____ 1s 3ms/step - accuracy: 0.4898 - loss: 1.7971 - val_accuracy: 0.5232 - val_loss: 1.6214

Epoch 27/100
280/280 _____ 1s 3ms/step - accuracy: 0.4927 - loss: 1.7892 - val_accuracy: 0.5084 - val_loss: 1.7604
Epoch 28/100
280/280 _____ 1s 3ms/step - accuracy: 0.5059 - loss: 1.7135 - val_accuracy: 0.5297 - val_loss: 1.5761
Epoch 29/100
280/280 _____ 1s 3ms/step - accuracy: 0.5120 - loss: 1.7257 - val_accuracy: 0.5660 - val_loss: 1.5620
Epoch 30/100
280/280 _____ 1s 3ms/step - accuracy: 0.5054 - loss: 1.7521 - val_accuracy: 0.5734 - val_loss: 1.4951
Epoch 31/100
280/280 _____ 1s 3ms/step - accuracy: 0.5107 - loss: 1.7382 - val_accuracy: 0.5762 - val_loss: 1.4681
Epoch 32/100
280/280 _____ 1s 3ms/step - accuracy: 0.5206 - loss: 1.6774 - val_accuracy: 0.5651 - val_loss: 1.5642
Epoch 33/100
280/280 _____ 1s 3ms/step - accuracy: 0.5121 - loss: 1.6899 - val_accuracy: 0.5864 - val_loss: 1.4386
Epoch 34/100
280/280 _____ 1s 4ms/step - accuracy: 0.5145 - loss: 1.7309 - val_accuracy: 0.5660 - val_loss: 1.4997
Epoch 35/100
280/280 _____ 1s 4ms/step - accuracy: 0.5278 - loss: 1.6613 - val_accuracy: 0.5493 - val_loss: 1.5735
Epoch 36/100
280/280 _____ 1s 3ms/step - accuracy: 0.5142 - loss: 1.7038 - val_accuracy: 0.5790 - val_loss: 1.4700
Epoch 37/100
280/280 _____ 1s 3ms/step - accuracy: 0.5110 - loss: 1.6807 - val_accuracy: 0.5669 - val_loss: 1.4794
Epoch 38/100
280/280 _____ 1s 3ms/step - accuracy: 0.5139 - loss: 1.6953 - val_accuracy: 0.5418 - val_loss: 1.5383
Epoch 39/100
280/280 _____ 1s 3ms/step - accuracy: 0.5181 - loss: 1.7153 - val_accuracy: 0.5483 - val_loss: 1.5613
Epoch 40/100
280/280 _____ 1s 3ms/step - accuracy: 0.5310 - loss: 1.6866 - val_accuracy: 0.4768 - val_loss: 1.8240
Epoch 41/100
280/280 _____ 1s 3ms/step - accuracy: 0.5160 - loss: 1.6787 - val_accuracy: 0.5697 - val_loss: 1.5213
Epoch 42/100
280/280 _____ 1s 3ms/step - accuracy: 0.5210 - loss: 1.6920 - val_accuracy: 0.5697 - val_loss: 1.4437
Epoch 43/100

280/280 _____ 1s 3ms/step - accuracy: 0.5323 - loss: 1.6526 - val_accuracy: 0.6004 - val_loss: 1.3942
Epoch 44/100
280/280 _____ 1s 3ms/step - accuracy: 0.5299 - loss: 1.6458 - val_accuracy: 0.5716 - val_loss: 1.4758
Epoch 45/100
280/280 _____ 1s 3ms/step - accuracy: 0.5225 - loss: 1.6548 - val_accuracy: 0.6180 - val_loss: 1.3594
Epoch 46/100
280/280 _____ 1s 3ms/step - accuracy: 0.5258 - loss: 1.6664 - val_accuracy: 0.5929 - val_loss: 1.4318
Epoch 47/100
280/280 _____ 1s 4ms/step - accuracy: 0.5386 - loss: 1.6365 - val_accuracy: 0.5855 - val_loss: 1.4409
Epoch 48/100
280/280 _____ 1s 4ms/step - accuracy: 0.5269 - loss: 1.6639 - val_accuracy: 0.5586 - val_loss: 1.5098
Epoch 49/100
280/280 _____ 1s 3ms/step - accuracy: 0.5284 - loss: 1.6289 - val_accuracy: 0.5706 - val_loss: 1.4711
Epoch 50/100
280/280 _____ 1s 3ms/step - accuracy: 0.5394 - loss: 1.6143 - val_accuracy: 0.5809 - val_loss: 1.5164
Epoch 51/100
280/280 _____ 1s 3ms/step - accuracy: 0.5463 - loss: 1.6055 - val_accuracy: 0.5920 - val_loss: 1.4253
Epoch 52/100
280/280 _____ 1s 3ms/step - accuracy: 0.5383 - loss: 1.6222 - val_accuracy: 0.5446 - val_loss: 1.6005
Epoch 53/100
280/280 _____ 1s 3ms/step - accuracy: 0.5267 - loss: 1.6245 - val_accuracy: 0.5929 - val_loss: 1.4495
Epoch 54/100
280/280 _____ 1s 3ms/step - accuracy: 0.5401 - loss: 1.6232 - val_accuracy: 0.5390 - val_loss: 1.6106
Epoch 55/100
280/280 _____ 1s 3ms/step - accuracy: 0.5300 - loss: 1.6121 - val_accuracy: 0.5799 - val_loss: 1.5223
Epoch 56/100
280/280 _____ 1s 3ms/step - accuracy: 0.5559 - loss: 1.5728 - val_accuracy: 0.5883 - val_loss: 1.4385
Epoch 57/100
280/280 _____ 1s 3ms/step - accuracy: 0.5274 - loss: 1.6281 - val_accuracy: 0.5697 - val_loss: 1.4959
Epoch 58/100
280/280 _____ 1s 3ms/step - accuracy: 0.5376 - loss: 1.6298 - val_accuracy: 0.6013 - val_loss: 1.4309
Epoch 59/100
280/280 _____ 1s 3ms/step - accuracy: 0.5347 - loss:

```
1.5895 - val_accuracy: 0.5446 - val_loss: 1.5820
Epoch 60/100
280/280 _____ 1s 4ms/step - accuracy: 0.5441 - loss:
1.5899 - val_accuracy: 0.5344 - val_loss: 1.7306
Epoch 61/100
280/280 _____ 1s 4ms/step - accuracy: 0.5449 - loss:
1.5814 - val_accuracy: 0.5344 - val_loss: 1.6502
Epoch 62/100
280/280 _____ 1s 3ms/step - accuracy: 0.5435 - loss:
1.5880 - val_accuracy: 0.6106 - val_loss: 1.3779
Epoch 63/100
280/280 _____ 1s 3ms/step - accuracy: 0.5408 - loss:
1.6221 - val_accuracy: 0.6283 - val_loss: 1.3354
Epoch 64/100
280/280 _____ 1s 3ms/step - accuracy: 0.5519 - loss:
1.5800 - val_accuracy: 0.5799 - val_loss: 1.4303
Epoch 65/100
280/280 _____ 1s 3ms/step - accuracy: 0.5515 - loss:
1.5636 - val_accuracy: 0.5901 - val_loss: 1.4392
Epoch 66/100
280/280 _____ 1s 3ms/step - accuracy: 0.5504 - loss:
1.5826 - val_accuracy: 0.6255 - val_loss: 1.3362
Epoch 67/100
280/280 _____ 1s 3ms/step - accuracy: 0.5596 - loss:
1.5671 - val_accuracy: 0.5883 - val_loss: 1.3653
Epoch 68/100
280/280 _____ 1s 3ms/step - accuracy: 0.5490 - loss:
1.5929 - val_accuracy: 0.5632 - val_loss: 1.5318
Epoch 69/100
280/280 _____ 1s 3ms/step - accuracy: 0.5533 - loss:
1.5576 - val_accuracy: 0.6385 - val_loss: 1.3029
Epoch 70/100
280/280 _____ 1s 3ms/step - accuracy: 0.5491 - loss:
1.5436 - val_accuracy: 0.5660 - val_loss: 1.4675
Epoch 71/100
280/280 _____ 1s 3ms/step - accuracy: 0.5665 - loss:
1.5138 - val_accuracy: 0.6478 - val_loss: 1.2804
Epoch 72/100
280/280 _____ 1s 3ms/step - accuracy: 0.5679 - loss:
1.5007 - val_accuracy: 0.5678 - val_loss: 1.4527
Epoch 73/100
280/280 _____ 1s 4ms/step - accuracy: 0.5560 - loss:
1.5417 - val_accuracy: 0.5892 - val_loss: 1.4303
Epoch 74/100
280/280 _____ 1s 4ms/step - accuracy: 0.5689 - loss:
1.5034 - val_accuracy: 0.6329 - val_loss: 1.3155
Epoch 75/100
280/280 _____ 1s 4ms/step - accuracy: 0.5674 - loss:
1.5270 - val_accuracy: 0.5855 - val_loss: 1.4251
```

Epoch 76/100
280/280 _____ 1s 3ms/step - accuracy: 0.5632 - loss: 1.5313 - val_accuracy: 0.5743 - val_loss: 1.4470
Epoch 77/100
280/280 _____ 1s 3ms/step - accuracy: 0.5578 - loss: 1.5369 - val_accuracy: 0.6087 - val_loss: 1.3413
Epoch 78/100
280/280 _____ 1s 3ms/step - accuracy: 0.5657 - loss: 1.5403 - val_accuracy: 0.6125 - val_loss: 1.3624
Epoch 79/100
280/280 _____ 1s 3ms/step - accuracy: 0.5620 - loss: 1.5404 - val_accuracy: 0.6255 - val_loss: 1.2999
Epoch 80/100
280/280 _____ 1s 3ms/step - accuracy: 0.5563 - loss: 1.5341 - val_accuracy: 0.5632 - val_loss: 1.4595
Epoch 81/100
280/280 _____ 1s 3ms/step - accuracy: 0.5536 - loss: 1.5348 - val_accuracy: 0.6180 - val_loss: 1.3455
Epoch 82/100
280/280 _____ 1s 3ms/step - accuracy: 0.5594 - loss: 1.5771 - val_accuracy: 0.6125 - val_loss: 1.3728
Epoch 83/100
280/280 _____ 1s 3ms/step - accuracy: 0.5587 - loss: 1.5202 - val_accuracy: 0.5985 - val_loss: 1.3959
Epoch 84/100
280/280 _____ 1s 3ms/step - accuracy: 0.5584 - loss: 1.5342 - val_accuracy: 0.6329 - val_loss: 1.3127
Epoch 85/100
280/280 _____ 1s 3ms/step - accuracy: 0.5663 - loss: 1.5147 - val_accuracy: 0.6041 - val_loss: 1.4134
Epoch 86/100
280/280 _____ 1s 4ms/step - accuracy: 0.5610 - loss: 1.5266 - val_accuracy: 0.6208 - val_loss: 1.3469
Epoch 87/100
280/280 _____ 1s 4ms/step - accuracy: 0.5673 - loss: 1.5364 - val_accuracy: 0.6348 - val_loss: 1.2913
Epoch 88/100
280/280 _____ 1s 3ms/step - accuracy: 0.5589 - loss: 1.5281 - val_accuracy: 0.6050 - val_loss: 1.3257
Epoch 89/100
280/280 _____ 1s 3ms/step - accuracy: 0.5618 - loss: 1.5191 - val_accuracy: 0.6097 - val_loss: 1.3614
Epoch 90/100
280/280 _____ 1s 3ms/step - accuracy: 0.5731 - loss: 1.5038 - val_accuracy: 0.6227 - val_loss: 1.3337
Epoch 91/100
280/280 _____ 1s 3ms/step - accuracy: 0.5712 - loss: 1.4986 - val_accuracy: 0.6208 - val_loss: 1.3585

Epoch 91: early stopping

Restoring model weights from the end of the best epoch: 71.

```
test_loss_word2vec, test_acc_word2vec =  
word2vec_dense_model.evaluate(X_test_w2v, y_test_processed, verbose=0)
```

```
y_pred_word2vec_proba = word2vec_dense_model.predict(X_test_w2v,  
verbose=0)
```

```
y_pred_word2vec = np.argmax(y_pred_word2vec_proba, axis=1)
```

```
f1_macro_word2vec = f1_score(y_test, y_pred_word2vec, average='macro')
```

```
print(f"\nTest Loss: {test_loss_word2vec:.4f}")
```

```
print(f"Test Accuracy: {test_acc_word2vec:.4f}")
```

```
print(f"F1-score (Macro): {f1_macro_word2vec:.4f}")
```

Test Loss: 1.3196

Test Accuracy: 0.6022

F1-score (Macro): 0.5788

```
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(history_word2vec.history['loss'], label='Train Loss')
```

```
plt.plot(history_word2vec.history['val_loss'], label='Val Loss')
```

```
plt.title('Word2Vec + Dense: Loss')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(history_word2vec.history['accuracy'], label='Train Acc')
```

```
plt.plot(history_word2vec.history['val_accuracy'], label='Val Acc')
```

```
plt.title('Word2Vec + Dense: Accuracy')
```

```
plt.xlabel('Epoch')
```

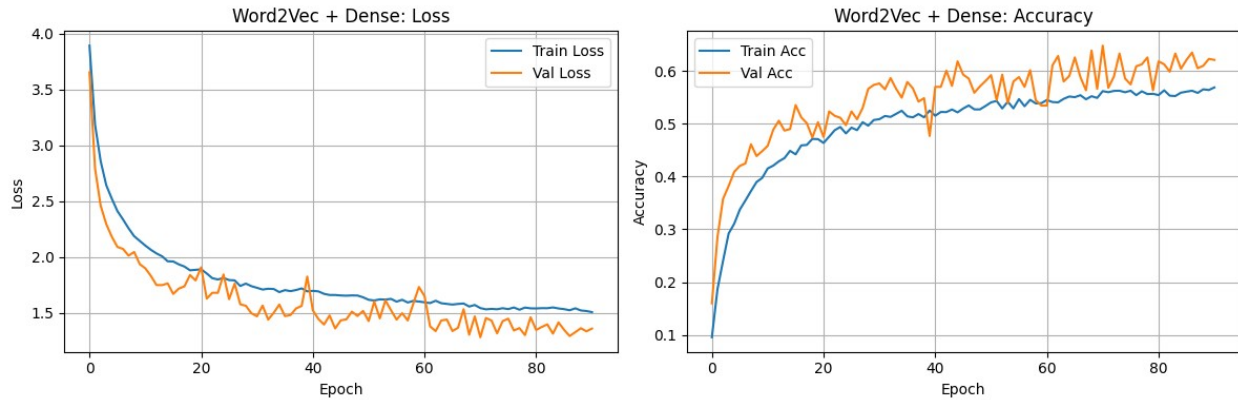
```
plt.ylabel('Accuracy')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



Nhiệm vụ 3: Mô hình Nâng cao (Embedding Pre-trained + LSTM)

```

vocab_size = 10000
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<UNK>")
tokenizer.fit_on_texts(X_train_text)

train_sequences = tokenizer.texts_to_sequences(X_train_text)
val_sequences = tokenizer.texts_to_sequences(X_val_text)
test_sequences = tokenizer.texts_to_sequences(X_test_text)

max_len = 50
X_train_pad = pad_sequences(train_sequences, maxlen=max_len,
padding='post')
X_val_pad = pad_sequences(val_sequences, maxlen=max_len,
padding='post')
X_test_pad = pad_sequences(test_sequences, maxlen=max_len,
padding='post')

print(f"\nVocabulary size: {len(tokenizer.word_index) + 1}")
print(f"Max sequence length: {max_len}")
print(f>Data train padding shape: {X_train_pad.shape}")
print(f>Data validation padding shape: {X_val_pad.shape}")
print(f>Data test padding shape: {X_test_pad.shape}")

word_index = tokenizer.word_index
vocab_size_actual = min(len(word_index) + 1, vocab_size)
embedding_dim = w2v_model.vector_size

embedding_matrix = np.zeros((vocab_size_actual, embedding_dim))

found_words = 0
for word, i in word_index.items():
    if i >= vocab_size:
        continue

```



```

if word in w2v_model.wv:
    embedding_matrix[i] = w2v_model.wv[word]
    found_words += 1

print(f"Found {found_words}/{vocab_size_actual} words in Word2Vec model")

```

```

Vocabulary size: 4265
Max sequence length: 50
Data train padding shape: (8954, 50)
Data validation padding shape: (1076, 50)
Data test padding shape: (1076, 50)
Found 4197/4265 words in Word2Vec model

```

```

lstm_pretrained_model = Sequential([
    Embedding(
        input_dim=vocab_size_actual,
        output_dim=embedding_dim,
        weights=[embedding_matrix],
        input_length=max_len,
        trainable=False
    ),
    LSTM(128, dropout=0.5, recurrent_dropout=0),
    Dense(num_classes, activation='softmax')
])

```

```

lstm_pretrained_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
lstm_pretrained_model.build(input_shape=(None, max_len))
```

```
lstm_pretrained_model.summary()
```

```
Model: "sequential_6"
```

Layer (type) Param #	Output Shape
embedding_3 (Embedding) 426,500	(None, 50, 100)
lstm_3 (LSTM) 117,248	(None, 128)

dense_15 (Dense)	(None, 64)	
8,256		

Total params: 552,004 (2.11 MB)

Trainable params: 125,504 (490.25 KB)

Non-trainable params: 426,500 (1.63 MB)

```
early_stop = EarlyStopping(monitor='val_loss', patience=20,
restore_best_weights=True)
```

```
history_lstm_pretrained = lstm_pretrained_model.fit(
    X_train_pad, y_train_processed,
    validation_data=(X_val_pad, y_val_processed),
    epochs=100,
    batch_size=256,
    callbacks=[early_stop]
)
```

Epoch 1/100

35/35 ————— 3s 34ms/step - accuracy: 0.0162 - loss: 4.1427 - val_accuracy: 0.0232 - val_loss: 4.0041

Epoch 2/100

35/35 ————— 0s 13ms/step - accuracy: 0.0293 - loss: 3.9785 - val_accuracy: 0.0539 - val_loss: 3.8434

Epoch 3/100

35/35 ————— 0s 11ms/step - accuracy: 0.0554 - loss: 3.8251 - val_accuracy: 0.0725 - val_loss: 3.6388

Epoch 4/100

35/35 ————— 0s 12ms/step - accuracy: 0.0679 - loss: 3.6581 - val_accuracy: 0.0781 - val_loss: 3.5580

Epoch 5/100

35/35 ————— 1s 14ms/step - accuracy: 0.0783 - loss: 3.5731 - val_accuracy: 0.0920 - val_loss: 3.4728

Epoch 6/100

35/35 ————— 1s 14ms/step - accuracy: 0.0788 - loss: 3.5166 - val_accuracy: 0.0883 - val_loss: 3.4302

Epoch 7/100

35/35 ————— 1s 14ms/step - accuracy: 0.0767 - loss: 3.5848 - val_accuracy: 0.0855 - val_loss: 3.4006

Epoch 8/100

35/35 ————— 1s 14ms/step - accuracy: 0.0820 - loss: 3.4790 - val_accuracy: 0.0939 - val_loss: 3.4055

Epoch 9/100

35/35 ————— 0s 11ms/step - accuracy: 0.0898 - loss:

3.4493 - val_accuracy: 0.0957 - val_loss: 3.3135
Epoch 10/100
35/35 _____ 0s 11ms/step - accuracy: 0.0650 - loss:
3.7876 - val_accuracy: 0.0651 - val_loss: 3.6345
Epoch 11/100
35/35 _____ 0s 11ms/step - accuracy: 0.0576 - loss:
3.7073 - val_accuracy: 0.0641 - val_loss: 3.5648
Epoch 12/100
35/35 _____ 0s 11ms/step - accuracy: 0.0609 - loss:
3.6251 - val_accuracy: 0.0762 - val_loss: 3.5540
Epoch 13/100
35/35 _____ 0s 12ms/step - accuracy: 0.0642 - loss:
3.6205 - val_accuracy: 0.0864 - val_loss: 3.4705
Epoch 14/100
35/35 _____ 0s 11ms/step - accuracy: 0.0643 - loss:
3.5752 - val_accuracy: 0.0725 - val_loss: 3.5627
Epoch 15/100
35/35 _____ 0s 11ms/step - accuracy: 0.0703 - loss:
3.5956 - val_accuracy: 0.0939 - val_loss: 3.4236
Epoch 16/100
35/35 _____ 1s 11ms/step - accuracy: 0.0794 - loss:
3.4716 - val_accuracy: 0.0846 - val_loss: 3.3479
Epoch 17/100
35/35 _____ 0s 11ms/step - accuracy: 0.0907 - loss:
3.4710 - val_accuracy: 0.0855 - val_loss: 3.3951
Epoch 18/100
35/35 _____ 0s 11ms/step - accuracy: 0.0836 - loss:
3.4683 - val_accuracy: 0.0874 - val_loss: 3.4227
Epoch 19/100
35/35 _____ 0s 11ms/step - accuracy: 0.0850 - loss:
3.4581 - val_accuracy: 0.0967 - val_loss: 3.3774
Epoch 20/100
35/35 _____ 0s 11ms/step - accuracy: 0.0868 - loss:
3.4805 - val_accuracy: 0.0874 - val_loss: 3.3617
Epoch 21/100
35/35 _____ 0s 11ms/step - accuracy: 0.0874 - loss:
3.4248 - val_accuracy: 0.0929 - val_loss: 3.3864
Epoch 22/100
35/35 _____ 0s 11ms/step - accuracy: 0.0812 - loss:
3.4897 - val_accuracy: 0.0799 - val_loss: 3.3742
Epoch 23/100
35/35 _____ 0s 11ms/step - accuracy: 0.0867 - loss:
3.4498 - val_accuracy: 0.0790 - val_loss: 3.4478
Epoch 24/100
35/35 _____ 0s 11ms/step - accuracy: 0.0839 - loss:
3.4928 - val_accuracy: 0.0892 - val_loss: 3.4927
Epoch 25/100
35/35 _____ 0s 11ms/step - accuracy: 0.0739 - loss:
3.5522 - val_accuracy: 0.0985 - val_loss: 3.4249

```

Epoch 26/100
35/35 _____ 0s 12ms/step - accuracy: 0.0874 - loss:
3.5102 - val_accuracy: 0.1013 - val_loss: 3.3524
Epoch 27/100
35/35 _____ 0s 11ms/step - accuracy: 0.0953 - loss:
3.4335 - val_accuracy: 0.0957 - val_loss: 3.4510
Epoch 28/100
35/35 _____ 0s 12ms/step - accuracy: 0.0759 - loss:
3.6408 - val_accuracy: 0.0781 - val_loss: 3.6099
Epoch 29/100
35/35 _____ 0s 11ms/step - accuracy: 0.0810 - loss:
3.6338 - val_accuracy: 0.0836 - val_loss: 3.5237

test_loss_lstm_pre, test_acc_lstm_pre =
lstm_pretrained_model.evaluate(X_test_pad, y_test_processed,
verbose=1)

y_pred_lstm_pre = np.argmax(lstm_pretrained_model.predict(X_test_pad),
axis=1)

f1_macro_lstm_pre = f1_score(y_test, y_pred_lstm_pre, average='macro')

print(f"\nTest Loss: {test_loss_lstm_pre:.3f}")
print(f"Test Accuracy: {test_acc_lstm_pre:.3f}")
print(f"F1-score (Macro): {f1_macro_lstm_pre:.3f}")

34/34 _____ 0s 5ms/step - accuracy: 0.0760 - loss:
3.4237
34/34 _____ 0s 6ms/step

Test Loss: 3.365
Test Accuracy: 0.089
F1-score (Macro): 0.034

plt.figure(figsize=(12, 4))

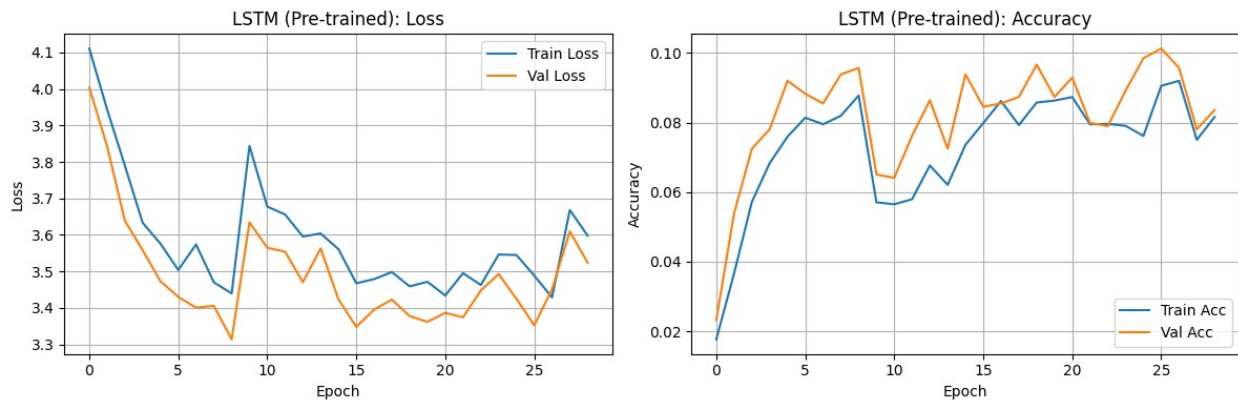
plt.subplot(1, 2, 1)
plt.plot(history_lstm_pretrained.history['loss'], label='Train Loss')
plt.plot(history_lstm_pretrained.history['val_loss'], label='Val
Loss')
plt.title('LSTM (Pre-trained): Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history_lstm_pretrained.history['accuracy'], label='Train
Acc')
plt.plot(history_lstm_pretrained.history['val_accuracy'], label='Val
Acc')

```

```
plt.title('LSTM (Pre-trained): Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Nhiệm vụ 4: Mô hình Nâng cao (Embedding học từ đầu + LSTM)

```
lstm_model = Sequential([
    Embedding(
        input_dim=vocab_size_actual,
        output_dim=100,
        input_length=max_len
    ),
    LSTM(128, dropout=0.5, recurrent_dropout=0),
    # Output layer
    Dense(num_classes, activation='softmax')
])

lstm_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

lstm_model.build(input_shape=(None, max_len))
lstm_model.summary()
```

Model: "sequential_7"

Layer (type) Param #	Output Shape
embedding_4 (Embedding) 426,500	(None, 50, 100)
lstm_4 (LSTM) 117,248	(None, 128)
dense_16 (Dense) 8,256	(None, 64)

Total params: 552,004 (2.11 MB)

Trainable params: 552,004 (2.11 MB)

Non-trainable params: 0 (0.00 B)

```
history_lstm = lstm_model.fit(  
    X_train_pad, y_train_processed,  
    validation_data=(X_val_pad, y_val_processed),  
    epochs=100,  
    batch_size=256,  
)
```

Epoch 1/100

35/35 ————— 3s 26ms/step - accuracy: 0.0164 - loss: 4.1525 - val_accuracy: 0.0177 - val_loss: 4.1326

Epoch 2/100

35/35 ————— 0s 12ms/step - accuracy: 0.0166 - loss: 4.1377 - val_accuracy: 0.0177 - val_loss: 4.1252

Epoch 3/100

35/35 ————— 0s 12ms/step - accuracy: 0.0162 - loss: 4.1287 - val_accuracy: 0.0177 - val_loss: 4.1271

Epoch 4/100

35/35 ————— 0s 11ms/step - accuracy: 0.0149 - loss: 4.1342 - val_accuracy: 0.0177 - val_loss: 4.1245

Epoch 5/100

35/35 ————— 0s 12ms/step - accuracy: 0.0169 - loss: 4.1277 - val_accuracy: 0.0177 - val_loss: 4.1248

Epoch 6/100

35/35 ————— 0s 11ms/step - accuracy: 0.0181 - loss:

4.1288 - val_accuracy: 0.0177 - val_loss: 4.1239
Epoch 7/100
35/35 _____ 0s 12ms/step - accuracy: 0.0126 - loss:
4.1269 - val_accuracy: 0.0177 - val_loss: 4.1255
Epoch 8/100
35/35 _____ 0s 12ms/step - accuracy: 0.0144 - loss:
4.1312 - val_accuracy: 0.0177 - val_loss: 4.1188
Epoch 9/100
35/35 _____ 0s 12ms/step - accuracy: 0.0184 - loss:
4.0841 - val_accuracy: 0.0177 - val_loss: 4.1283
Epoch 10/100
35/35 _____ 0s 12ms/step - accuracy: 0.0168 - loss:
4.1296 - val_accuracy: 0.0177 - val_loss: 4.1246
Epoch 11/100
35/35 _____ 0s 12ms/step - accuracy: 0.0188 - loss:
4.1309 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 12/100
35/35 _____ 1s 12ms/step - accuracy: 0.0156 - loss:
4.1266 - val_accuracy: 0.0177 - val_loss: 4.1239
Epoch 13/100
35/35 _____ 0s 11ms/step - accuracy: 0.0150 - loss:
4.1290 - val_accuracy: 0.0177 - val_loss: 4.1241
Epoch 14/100
35/35 _____ 0s 12ms/step - accuracy: 0.0179 - loss:
4.1333 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 15/100
35/35 _____ 0s 12ms/step - accuracy: 0.0157 - loss:
4.1345 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 16/100
35/35 _____ 1s 12ms/step - accuracy: 0.0151 - loss:
4.1286 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 17/100
35/35 _____ 0s 12ms/step - accuracy: 0.0171 - loss:
4.1319 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 18/100
35/35 _____ 0s 12ms/step - accuracy: 0.0182 - loss:
4.1289 - val_accuracy: 0.0177 - val_loss: 4.1240
Epoch 19/100
35/35 _____ 0s 12ms/step - accuracy: 0.0216 - loss:
4.1299 - val_accuracy: 0.0177 - val_loss: 4.1235
Epoch 20/100
35/35 _____ 0s 12ms/step - accuracy: 0.0157 - loss:
4.1254 - val_accuracy: 0.0177 - val_loss: 4.1240
Epoch 21/100
35/35 _____ 0s 12ms/step - accuracy: 0.0173 - loss:
4.1294 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 22/100
35/35 _____ 0s 12ms/step - accuracy: 0.0155 - loss:
4.1280 - val_accuracy: 0.0177 - val_loss: 4.1238

Epoch 23/100
35/35 _____ 1s 16ms/step - accuracy: 0.0174 - loss: 4.1301 - val_accuracy: 0.0177 - val_loss: 4.1235

Epoch 24/100
35/35 _____ 1s 15ms/step - accuracy: 0.0169 - loss: 4.1261 - val_accuracy: 0.0251 - val_loss: 4.0660

Epoch 25/100
35/35 _____ 1s 15ms/step - accuracy: 0.0213 - loss: 4.0453 - val_accuracy: 0.0177 - val_loss: 4.1293

Epoch 26/100
35/35 _____ 1s 14ms/step - accuracy: 0.0161 - loss: 4.1366 - val_accuracy: 0.0177 - val_loss: 4.1241

Epoch 27/100
35/35 _____ 0s 11ms/step - accuracy: 0.0139 - loss: 4.1322 - val_accuracy: 0.0177 - val_loss: 4.1238

Epoch 28/100
35/35 _____ 0s 12ms/step - accuracy: 0.0170 - loss: 4.1289 - val_accuracy: 0.0177 - val_loss: 4.1238

Epoch 29/100
35/35 _____ 0s 11ms/step - accuracy: 0.0175 - loss: 4.1290 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 30/100
35/35 _____ 0s 11ms/step - accuracy: 0.0168 - loss: 4.1305 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 31/100
35/35 _____ 0s 11ms/step - accuracy: 0.0161 - loss: 4.1301 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 32/100
35/35 _____ 0s 12ms/step - accuracy: 0.0161 - loss: 4.1284 - val_accuracy: 0.0177 - val_loss: 4.1236

Epoch 33/100
35/35 _____ 0s 12ms/step - accuracy: 0.0171 - loss: 4.1287 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 34/100
35/35 _____ 0s 12ms/step - accuracy: 0.0166 - loss: 4.1315 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 35/100
35/35 _____ 0s 11ms/step - accuracy: 0.0162 - loss: 4.1297 - val_accuracy: 0.0177 - val_loss: 4.1236

Epoch 36/100
35/35 _____ 0s 12ms/step - accuracy: 0.0171 - loss: 4.1287 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 37/100
35/35 _____ 0s 12ms/step - accuracy: 0.0189 - loss: 4.1315 - val_accuracy: 0.0177 - val_loss: 4.1238

Epoch 38/100
35/35 _____ 0s 11ms/step - accuracy: 0.0165 - loss: 4.1298 - val_accuracy: 0.0177 - val_loss: 4.1237

Epoch 39/100

35/35 _____ 0s 12ms/step - accuracy: 0.0160 - loss: 4.1326 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 40/100

35/35 _____ 0s 11ms/step - accuracy: 0.0177 - loss: 4.1264 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 41/100

35/35 _____ 0s 11ms/step - accuracy: 0.0147 - loss: 4.1337 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 42/100

35/35 _____ 0s 12ms/step - accuracy: 0.0152 - loss: 4.1319 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 43/100

35/35 _____ 0s 11ms/step - accuracy: 0.0177 - loss: 4.1291 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 44/100

35/35 _____ 0s 12ms/step - accuracy: 0.0124 - loss: 4.1292 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 45/100

35/35 _____ 0s 12ms/step - accuracy: 0.0206 - loss: 4.1299 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 46/100

35/35 _____ 1s 12ms/step - accuracy: 0.0150 - loss: 4.1285 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 47/100

35/35 _____ 0s 11ms/step - accuracy: 0.0134 - loss: 4.1295 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 48/100

35/35 _____ 0s 12ms/step - accuracy: 0.0173 - loss: 4.1280 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 49/100

35/35 _____ 0s 13ms/step - accuracy: 0.0148 - loss: 4.1296 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 50/100

35/35 _____ 1s 15ms/step - accuracy: 0.0158 - loss: 4.1323 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 51/100

35/35 _____ 1s 14ms/step - accuracy: 0.0170 - loss: 4.1277 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 52/100

35/35 _____ 1s 15ms/step - accuracy: 0.0193 - loss: 4.1337 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 53/100

35/35 _____ 1s 13ms/step - accuracy: 0.0169 - loss: 4.1308 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 54/100

35/35 _____ 0s 11ms/step - accuracy: 0.0161 - loss: 4.1248 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 55/100

35/35 _____ 0s 12ms/step - accuracy: 0.0160 - loss:

4.1306 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 56/100
35/35 _____ 0s 11ms/step - accuracy: 0.0164 - loss:
4.1322 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 57/100
35/35 _____ 0s 12ms/step - accuracy: 0.0162 - loss:
4.1260 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 58/100
35/35 _____ 0s 12ms/step - accuracy: 0.0149 - loss:
4.1256 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 59/100
35/35 _____ 0s 11ms/step - accuracy: 0.0172 - loss:
4.1266 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 60/100
35/35 _____ 0s 11ms/step - accuracy: 0.0158 - loss:
4.1285 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 61/100
35/35 _____ 0s 12ms/step - accuracy: 0.0188 - loss:
4.1271 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 62/100
35/35 _____ 0s 12ms/step - accuracy: 0.0117 - loss:
4.1302 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 63/100
35/35 _____ 0s 12ms/step - accuracy: 0.0166 - loss:
4.1274 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 64/100
35/35 _____ 0s 12ms/step - accuracy: 0.0221 - loss:
4.1273 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 65/100
35/35 _____ 0s 11ms/step - accuracy: 0.0159 - loss:
4.1291 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 66/100
35/35 _____ 0s 11ms/step - accuracy: 0.0167 - loss:
4.1271 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 67/100
35/35 _____ 0s 12ms/step - accuracy: 0.0163 - loss:
4.1271 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 68/100
35/35 _____ 0s 12ms/step - accuracy: 0.0152 - loss:
4.1284 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 69/100
35/35 _____ 0s 12ms/step - accuracy: 0.0151 - loss:
4.1287 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 70/100
35/35 _____ 0s 11ms/step - accuracy: 0.0167 - loss:
4.1285 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 71/100
35/35 _____ 0s 11ms/step - accuracy: 0.0142 - loss:
4.1292 - val_accuracy: 0.0177 - val_loss: 4.1236

Epoch 72/100
35/35 _____ 0s 12ms/step - accuracy: 0.0176 - loss: 4.1272 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 73/100
35/35 _____ 0s 11ms/step - accuracy: 0.0122 - loss: 4.1280 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 74/100
35/35 _____ 0s 12ms/step - accuracy: 0.0168 - loss: 4.1289 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 75/100
35/35 _____ 0s 11ms/step - accuracy: 0.0159 - loss: 4.1289 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 76/100
35/35 _____ 0s 11ms/step - accuracy: 0.0185 - loss: 4.1302 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 77/100
35/35 _____ 1s 15ms/step - accuracy: 0.0161 - loss: 4.1260 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 78/100
35/35 _____ 1s 15ms/step - accuracy: 0.0201 - loss: 4.1300 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 79/100
35/35 _____ 1s 14ms/step - accuracy: 0.0153 - loss: 4.1277 - val_accuracy: 0.0177 - val_loss: 4.1235
Epoch 80/100
35/35 _____ 1s 16ms/step - accuracy: 0.0176 - loss: 4.1286 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 81/100
35/35 _____ 0s 11ms/step - accuracy: 0.0149 - loss: 4.1290 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 82/100
35/35 _____ 0s 11ms/step - accuracy: 0.0162 - loss: 4.1282 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 83/100
35/35 _____ 0s 11ms/step - accuracy: 0.0139 - loss: 4.1292 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 84/100
35/35 _____ 0s 11ms/step - accuracy: 0.0170 - loss: 4.1280 - val_accuracy: 0.0177 - val_loss: 4.1235
Epoch 85/100
35/35 _____ 0s 12ms/step - accuracy: 0.0161 - loss: 4.1291 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 86/100
35/35 _____ 0s 11ms/step - accuracy: 0.0163 - loss: 4.1235 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 87/100
35/35 _____ 0s 12ms/step - accuracy: 0.0165 - loss: 4.1257 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 88/100

```

35/35 _____ 0s 11ms/step - accuracy: 0.0160 - loss:
4.1305 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 89/100
35/35 _____ 0s 12ms/step - accuracy: 0.0164 - loss:
4.1247 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 90/100
35/35 _____ 0s 11ms/step - accuracy: 0.0163 - loss:
4.1267 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 91/100
35/35 _____ 0s 11ms/step - accuracy: 0.0140 - loss:
4.1261 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 92/100
35/35 _____ 1s 12ms/step - accuracy: 0.0174 - loss:
4.1259 - val_accuracy: 0.0177 - val_loss: 4.1238
Epoch 93/100
35/35 _____ 0s 11ms/step - accuracy: 0.0172 - loss:
4.1277 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 94/100
35/35 _____ 0s 12ms/step - accuracy: 0.0159 - loss:
4.1263 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 95/100
35/35 _____ 0s 11ms/step - accuracy: 0.0138 - loss:
4.1278 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 96/100
35/35 _____ 0s 11ms/step - accuracy: 0.0170 - loss:
4.1268 - val_accuracy: 0.0177 - val_loss: 4.1236
Epoch 97/100
35/35 _____ 0s 12ms/step - accuracy: 0.0164 - loss:
4.1302 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 98/100
35/35 _____ 0s 11ms/step - accuracy: 0.0161 - loss:
4.1260 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 99/100
35/35 _____ 1s 12ms/step - accuracy: 0.0160 - loss:
4.1270 - val_accuracy: 0.0177 - val_loss: 4.1237
Epoch 100/100
35/35 _____ 0s 11ms/step - accuracy: 0.0151 - loss:
4.1257 - val_accuracy: 0.0177 - val_loss: 4.1236

```

```

test_loss_lstm, test_acc_lstm = lstm_model.evaluate(X_test_pad,
y_test_processed, verbose=1)

```

```

y_pred_lstm = np.argmax(lstm_model.predict(X_test_pad), axis=1)

```

```

f1_macro_lstm = f1_score(y_test, y_pred_lstm, average='macro')

```

```

print(f"\nTest Loss: {test_loss_lstm:.3f}")
print(f"Test Accuracy: {test_acc_lstm:.3f}")
print(f"F1-score (Macro): {f1_macro_lstm:.3f}")

```

```
34/34 ————— 0s 5ms/step - accuracy: 0.0061 - loss: 4.1532  
34/34 ————— 0s 6ms/step
```

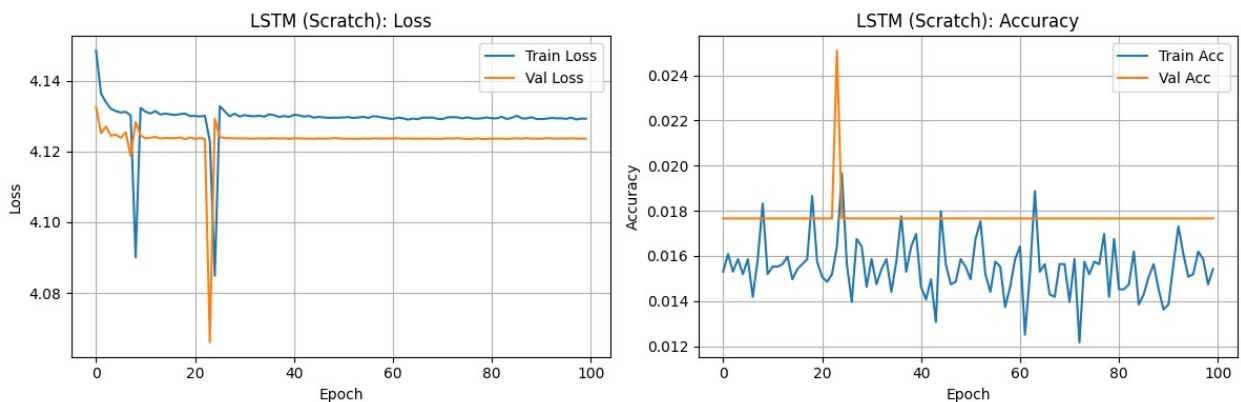
```
Test Loss: 4.124  
Test Accuracy: 0.018  
F1-score (Macro): 0.001
```

```
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)  
plt.plot(history_lstm.history['loss'], label='Train Loss')  
plt.plot(history_lstm.history['val_loss'], label='Val Loss')  
plt.title('LSTM (Scratch): Loss')  
plt.xlabel('Epoch')  
plt.ylabel('Loss')  
plt.legend()  
plt.grid(True)
```

```
plt.subplot(1, 2, 2)  
plt.plot(history_lstm.history['accuracy'], label='Train Acc')  
plt.plot(history_lstm.history['val_accuracy'], label='Val Acc')  
plt.title('LSTM (Scratch): Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.grid(True)
```

```
plt.tight_layout()  
plt.show()
```



Nhiệm vụ 5: Đánh giá, So sánh và Phân tích

1. So sánh định lượng

```
results = pd.DataFrame({
    'Pipeline': [
        'TF-IDF + Logistic Regression',
        'Word2Vec (Avg) + Dense',
        'Embedding (Pre-trained) + LSTM',
        'Embedding (Scratch) + LSTM'
    ],
    'Accuracy': [
        accuracy_tfidf_log_reg,
        test_acc_word2vec,
        test_acc_lstm_pre,
        test_acc_lstm
    ],
    'F1-score (Macro)': [
        f1_macro_tfidf_log_reg,
        f1_macro_word2vec,
        f1_macro_lstm_pre,
        f1_macro_lstm
    ],
    'Test Loss': [
        'N/A',
        f"{test_loss_word2vec:.3f}",
        f"{test_loss_lstm_pre:.3f}",
        f"{test_loss_lstm:.3f}"
    ]
})
```

```
print("="*80)
print("BẢNG SO SÁNH KẾT QUẢ CÁC MÔ HÌNH")
print("="*80)
print(results.to_string())
print("="*80)
```

```
=====
=====
BẢNG SO SÁNH KẾT QUẢ CÁC MÔ HÌNH
=====
=====
```

	Pipeline	Accuracy	F1-score (Macro)	Test
Loss				
0	TF-IDF + Logistic Regression	0.833643	0.830032	
N/A				
1	Word2Vec (Avg) + Dense	0.602230	0.578762	
1.320				
2	Embedding (Pre-trained) + LSTM	0.089219	0.034037	

```

3.365
3      Embedding (Scratch) + LSTM    0.017658          0.000542
4.124
=====
=====

```

2. Phân tích định tính

a. Lấy dự đoán từ cả 4 mô hình. b. So sánh với nhãn thật

```

test_difficult_sentences = [
    "can you remind me to not call my mom",
    "is it going to be sunny or rainy tomorrow",
    "find a flight from new york to london but not through paris"
]

labels = ["reminder_create", "weather_query", "flight_search"]

print("="*80)
print("PHÂN TÍCH ĐỊNH TÍNH")
print("="*80)

for idx, (sentence, label_) in enumerate(zip(test_difficult_sentences,
labels), 1):
    print(f"\n{idx}. Test difficult sentence: '{sentence}'")
    print("-" * 80)

    # TF-IDF + LR
    pred_tfidf = tfidf_log_reg_pipeline.predict([sentence])[0]
    print(f"TF-IDF + LR: {label_encoder.classes_[pred_tfidf]}")

    # Word2Vec + Dense
    sentence_vec = sentence_to_avg_vector(sentence,
w2v_model).reshape(1, -1)
    pred_w2v = word2vec_dense_model.predict(sentence_vec, verbose=0)
    pred_w2v_class = label_encoder.classes_[np.argmax(pred_w2v)]
    print(f"Word2Vec + Dense: {pred_w2v_class}")

    # LSTM Pre-trained
    sentence_seq = tokenizer.texts_to_sequences([sentence])
    sentence_pad = pad_sequences(sentence_seq, maxlen=max_len,
padding='post')
    pred_lstm_pre = lstm_pretrained_model.predict(sentence_pad,
verbose=0)
    pred_lstm_pre_class =
label_encoder.classes_[np.argmax(pred_lstm_pre)]
    print(f"LSTM (Pre-trained): {pred_lstm_pre_class}")

    # LSTM Scratch

```

```

pred_lstm = lstm_model.predict(sentence_pad, verbose=0)
pred_lstm_class = label_encoder.classes_[np.argmax(pred_lstm)]
print(f"LSTM (Scratch): {pred_lstm_class}")

print(f"Label: {label_}")

```

PHÂN TÍCH ĐỊNH TÍNH

1. Test difficult sentence: 'can you remind me to not call my mom'

TF-IDF + LR:calendar_set
Word2Vec + Dense: email_query
LSTM (Pre-trained): play_music
LSTM (Scratch): qa_definition
Label: reminder_create

2. Test difficult sentence: 'is it going to be sunny or rainy tomorrow'

TF-IDF + LR:weather_query
Word2Vec + Dense: weather_query
LSTM (Pre-trained): play_podcasts
LSTM (Scratch): qa_definition
Label: weather_query

3. Test difficult sentence: 'find a flight from new york to london but not through paris'

TF-IDF + LR:transport_query
Word2Vec + Dense: transport_query
LSTM (Pre-trained): transport_ticket
LSTM (Scratch): qa_definition
Label: flight_search

c. Nhận xét xem mô hình nào (đặc biệt là các mô hình LSTM) đưa ra dự đoán chính xác hơn và tại sao. Liệu khả năng xử lý chuỗi có giúp chúng hiểu đúng ý định trong các câu phức tạp này không

Mô hình Phi-Chuỗi (TF-IDF + LR, Word2Vec + Dense):

- Ở Câu 2, chúng hoạt động tốt hơn 2 mô hình LSTM. Điều này cho thấy với câu hỏi thời tiết đơn giản, sự hiện diện của các từ khóa như "sunny", "rainy", "tomorrow"

là đủ để phân loại chính xác, và mô hình TF-IDF dựa trên tần suất từ khóa đã làm việc hiệu quả.

- Ở Câu 1 và Câu 3, chúng không hoàn toàn chính xác nhưng ở Câu 3 đưa ra nhãn gần đúng (`transport_query`).

Mô hình Xử lý Chuỗi (LSTM):

- LSTM (Pre-trained) và LSTM (Scratch) có hiệu suất kém nhất trong cả ba trường hợp, thường xuyên đưa ra các nhãn sai lệch hoàn toàn (`play_music`, `play_podcasts`, `qa_definition`).
- Đặc biệt, LSTM (Scratch) dường như mặc định dự đoán `qa_definition` cho hầu hết các câu khó, có thể là do bộ dữ liệu training nhỏ hoặc lỗi trong quá trình huấn luyện khiến mô hình chưa học được mối quan hệ chuỗi từ ngữ hiệu quả.

Khả năng xử lý chuỗi của các mô hình LSTM hiện tại đã không giúp chúng hiểu đúng ý định trong ba câu phức tạp. Các mô hình LSTM (đặc biệt là bản huấn luyện từ đầu) trong thử nghiệm này không được huấn luyện đủ tốt hoặc dữ liệu huấn luyện chưa đủ để xử lý các câu có cấu trúc phức tạp.